

A Framework for Integrated Behaviour Driven Development and Model Based Feature Design and Verification

Sandeep Sandhu^{1,3}, Sulakshan Rajendran³, Fernando Sarracini Jr², and Morteza Soleimani³

¹Ford Motor Company, U.K.

²Ford Motor Company, Brazil

³University of Warwick, Warwick Manufacturing Group (WMG), U.K.

Keywords: Software Defined Vehicles, Requirements Engineering, Behaviour Driven Development, Model Based Systems Engineering, Automated Feature Testing.

Abstract: The rapid evolution of Software Defined Vehicles (SDVs) has shifted automotive innovation toward agile, software intensive features. These features are often distributed across zonal controllers or domain controllers and involve million lines of code. This makes traditional systems engineering and manual testing insufficient and impossible to test the features comprehensively. This paper proposes a new integrated framework which combines both the Behavior Driven Development (BDD) and Model Based System Engineering (MBSE) methodologies via multi loops to bridge the semantic gap between stakeholder and technical implementation following systems engineering V-cycle. By utilising Gherkin-based machine executable specifications with Behavioural Programming to drive formal MBSE Systems Modeling Language (SysML) behaviour models, the framework enables continuous verification upfront and throughout the V-cycle. This enhances requirements/specifications correctness and robustness within the feature software development cycle. The framework is evaluated using an automotive Remote keyless Entry (RKE) case study. Results demonstrate framework implementation and simulation of requirements and thus reducing test and development time.

1 INTRODUCTION

The global automotive industry is witnessing a significant expansion in software and electronic content, which is projected to achieve the second-highest share in the market revenue by 2030 (*Automotive Software and Electronics 2030*, 2030). Users increasingly expect vehicles defined by digital features, seamless connectivity, and higher levels of autonomy (Apostu et al., 2019). To meet these demands, Original Equipment Manufacturers (OEMs) are transforming traditional vehicle systems into integrated "systems of systems" where software defines the vehicle's lifecycle and value (Apostu et al., 2019).

A successful SDV transition requires decoupling of hardware and software development cycles, allowing software teams to iterate and deploy updates more independently. However, this decoupling introduces a significant challenge. Current vehicle software architectures are estimated to contain approximately 1.0×10^8 lines of code, a figure projected to triple by 2030 as feature complexity

increases (Aktas 2023). In such an environment, traditional document-centric requirements engineering and manual testing processes are not feasible.

The current feature design practices often rely on expensive physical prototype testing as the final validation step. Requirement-related issues such as semantic ambiguity or syntax errors often propagate through the development lifecycle and undetected till later validation stages. While Model-Based Systems Engineering (MBSE) tools like SysML provides system architecture structural rigor, it lacks a mechanism for early automated verification of stakeholder intent. To bridge this gap, this paper proposes an integrated framework that combines Behaviour-Driven Development (BDD) with MBSE methodologies. By transforming static, textual requirements to machine-readable executable specifications, the framework enables the "shift-left" of verification activities. This approach ensures requirements correctness and robustness.

The remainder of this paper is organized as follows: Section 2 reviews the theoretical foundations

of MBSE, BDD, and automated testing; Section 3 details the proposed structured framework based on BDD and MBSE methodology; Section 4 presents the results and analysis of the RKE feature case study for Phase 0 and Phase1, and finally Section 5 concludes with limitations and recommendations for future work.

2 BACKGROUND AND RELATED WORK

Current literature survey identifies distinct but complementary technologies that support the modern automotive development cycle.

2.1 Requirements Engineering (RE)

Requirements Engineering (RE) serves as the conjunction of requirements elicitation and requirements management, typically following the V-model (Graessler & Hentze, 2020). Main challenge in traditional RE is the reliance on natural language (textual) documentation, which is prone to ambiguity, lack of context, and difficult to test at scale (Staron, 2019).

As specification grows to around 100,000 pages or more, manual review can become challenging and more error prone. Researchers have implemented Natural Language Processing (NLP) algorithms to automatically verify requirements against standardized templates like EARS, thus significantly improving requirements robustness (Júnior et al., 2024).

2.2 Model Based Systems Engineering (MBSE)

MBSE utilises formal models and typically support the specification, system architecture design, and verification across the complete development cycle. In the automotive domain, a central benefit of MBSE is the establishment of a "single source of truth" where requirements are linked to logical system design and specific test cases.

The formalism helps identify inconsistencies much earlier than document centric methods, facilitating early impact analysis and verification through simulation (Wiecher et al., 2020).

2.3 Automated versus Manual Testing

Automated testing utilises software tools to execute tests, whereas manual testing relies on human input and verification. While the initial creation of

automated tests is more time-consuming than manual testing, the cost per test decreases significantly as repetitions increase, eventually becoming more cost-effective than manual methods (Bajer et al., 2015), illustrated in Figure 1. Notably, industry leaders have gained advantages by shifting testing "upfront" and utilizing test automation to achieve shorter development cycles (Ali Rizvi et al., 2025).

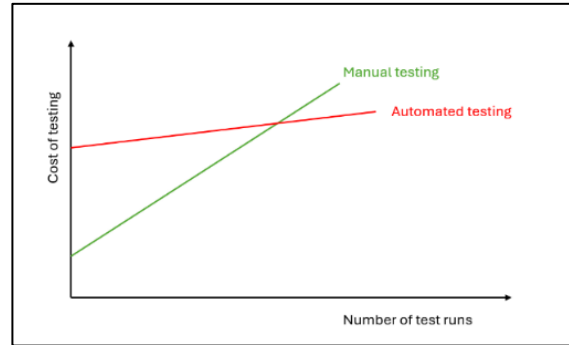


Figure 1: Manual versus Automated testing, drawn from (Bajer et al., 2025).

2.4 Behaviour Driven Development (BDD)

BDD is a "test-first" agile process that specifies system behaviour through examples utilising syntax "Given-When-Then" (Cucumber, 2025). This Domain Specific Language (DSL) bridges the gap between technical and non-technical stakeholders, enabling collaboration by fostering shared understanding of feature intent (Zameni et al., 2023; Gill 2022). The process fundamentally involves three key phases: discovery, formulation and automation.

2.5 Behavioural Programming (BP)

Behavioural Programming (BP) (Harel et al., 2012a) is a declarative paradigm for modeling reactive systems by coordinating event-driven behaviours known as b-threads. BP technique enables a formal synthesis of requirements into executable models, bridging the gap between high level specifications and implementations. At system level it is applied for verification of complex scenario-based systems (Wiecher et al., 2020).

2.6 Model Based Testing (MBT)

The testing of model-based systems is done using Model Based Testing (MBT) which is applied in many industrial domains, including automotive (Zander et al., 2017). In MBT, the functional or

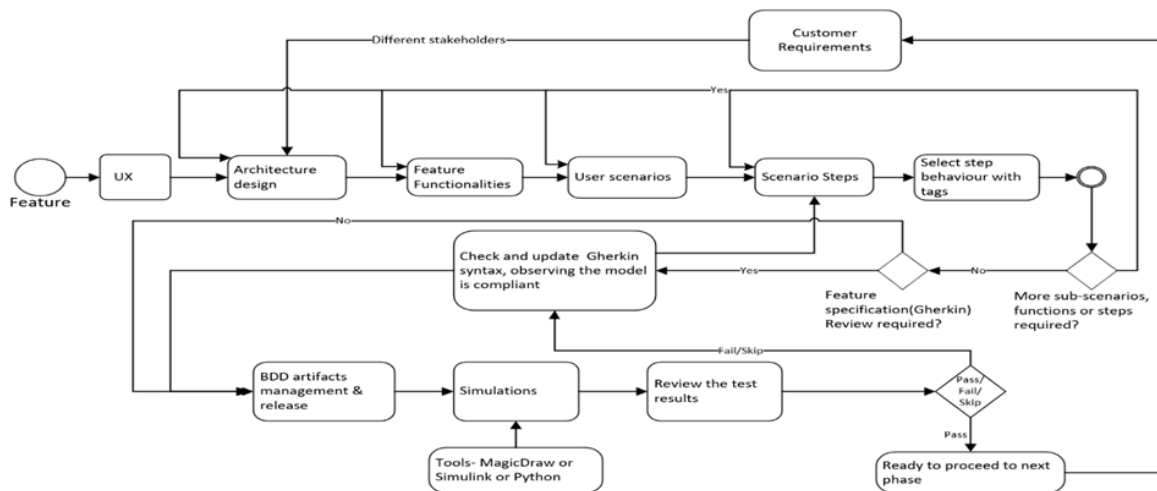


Figure 2: Proposed structured framework with integration of BDD and MBSE process for design, and testing of a feature; UX: User Experience.

behavioural aspects of the system are under test. MBT has key benefits of requirements to test cases traceability, high fault detection ratio, cost test effort reduction, and managing requirements changes to the test cases update (Petrenko et al., 2012; Utting & Legeard, 2007).

2.7 Integration of BDD and MBSE

Integrating BDD with MBSE allows for "Scenarios in the Loop" (SCIL), where user-oriented Gherkin scenarios are used to validate formal system models continuously (Wiecher et al., 2020). In their architecture Behavioural Programming (BP) (Harel et al., 2012b) is utilised as a formal technique for requirements-to-code synthesis. Despite these challenges are there, combining the methodologies can enhance early fault detection and reduce development costs by approximately 30% (Aktas 2023; Zamani et al. 2023).

3 INTEGRATED FRAMEWORK

In this section, the framework for Integrated MBSE and BDD following agile software development practices are described. The framework overview is illustrated in Figure 2. It consists of three primary phases: Feature Modeling, Behaviour specification and Verification via multi loops, which promotes agile development process. The framework utilizes MBT principles in Phase 1 to execute BDD scenarios against the SysML state machines ensuring the design model remains a 'single source of truth' for both development and verification.

3.1 Feature Modeling

The initial phase focuses on eliciting stakeholder needs. User Experience (UX) goals are abstracted into functional use cases and are formalised using Gherkin syntax (Given-When-Then). Thus, ensuring user experience and stakeholder requirements are high level human readable and machine executable. Simultaneously, a system architecture design interfaces between the user inputs, vehicle, environment, and feature settings. Feature functions are then decomposed into specific user scenarios to generate the BDD feature specification file.

3.2 Behavioural Specification

A step file is defined based on Gherkin feature file and at later phases is refined with signal details and enumerations from the system architecture design. A behavioural model (State Machine or Activity Diagram) is developed to represent system states and events. A critical "Compliance Check" verifies this model against the Gherkin scenarios to ensure architectural alignment. Finally, all artifacts are released to a version-controlled repository (e.g., GitHub). Checks/verifications are proposed based on Behavioural Programming (BP) technique.

3.3 Requirements Verification and Loops

The verification phase utilizes execution of BDD simulation robustly against the behavioural models. Upon completion, a test report analysis is conducted to review the pass/fail/skip results and test coverage.

If the feature-level tests pass, the methodology scales to next level of development and testing activities.

To maintain continuous alignment throughout the development lifecycle, the framework employs two separate feedback mechanisms:

- The Inner Loop: Applied for technical verification (Model-in-the-Loop).
- The Outer Loop: Links verification results back to the original customer requirements (Requirement-in-the-Loop)

This integrated approach enhances requirement robustness and allows for generation of diverse test scenarios that are repeatable and quickly executable.

4 CASE STUDY: RESULTS AND ANALYSIS

This section presents the results of applying framework to the design and early-phase verification of an automotive “Remote Keyless Entry” (RKE) feature.

Following top-down approach, the framework is applied for two phases to demonstrate the application.

- Phase 0 (Feature Concept)- The focus is on user experience requirement verification to ensure we are building the right thing from beginning through MATLAB-based simple model. This enables to develop effective stakeholder communication and high-level behavioural understanding of the system
- Phase 1 (Feature System Design)- The focus is on formalising the system design applying MBSE-based structural/behavioural modelling and scenario-driven simulation.

4.1 Phase 0: Feature Discovery and Requirements Verification

In Phase 0 stakeholders requirements understanding and collaboration is a priority and so high-level model is utilized between business stakeholders and technical teams.

4.1.1 User Experience and Requirement Elicitation

The initial step involved capturing the user experience (UX) of locking/unlocking interactions using remote key fobs. The interactions provided the foundation for identifying user expectations including safety, reliability, timely response, emergency unlocking, and consistent behaviour

across approach, driving, and crash events. From the UX analysis, some of the requirements formalised, are:-

- Remote Locking/Unlocking (RQ01)
- Customisable Unlocking Behaviour with Single-Point Entry (SPE) and Multi-Point Entry (MPE) (RQ02)
- Emergency Unlocking after a crash (RQ03)
- Auto-Locking during driving above 15 mph (RQ04), updated per functional safety review and ISO 26262 (*ISO 26262-1*, 2018) compliance.
- After crash clears, lock and unlock button works (RQ05)

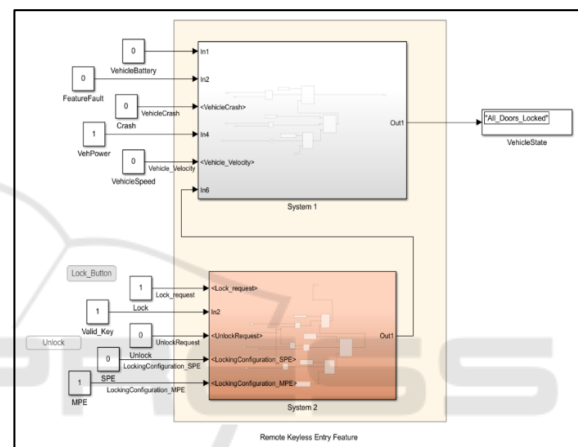


Figure 3: Phase0: Remote Keyless Entry (RKE) feature Simulink model. In this, left hand side is input, and the right-hand side is output in string format.

4.1.2 Feature Model

A high-level model is created in Simulink to abstract the intended RKE behaviour, Figure 3. The model describes the user interactions and serves as a communication artifact for aligning product owners, UX, and functional safety stakeholders around inputs, outputs, and priority logic. Notably,

- Critical requirements RQ03, RQ04 and RQ05 were implemented with prioritisation logic, ensuring safety-related unlocking/locking behaviour that overrides other requests.
- Inputs such as lock/unlock request, crash events, vehicle speed, service configurations (SPE/MPE) are considered along with the outputs representing the door states.

This Simulink model functioned as a behavioural “black box” for high level feature system logic.

4.1.3 Simulation Results

```

1 Feature: Remote Keyless Entry Door Locking and Unlocking.
2
3 @Scenario: @Remote_Lock
4 Scenario: While the vehicle is stationary, when the user presses the lock button the feature should lock all the doors
5 Given the vehicle is stationary
6 And the vehicle crash status is "no"
7 And the vehicle battery is "ok"
8 And the unlock button is released
9 And the user key is valid
10
11 When the user has pressed the lock button
12
13 Then all the doors should be LOCKED
14

```

Figure 4: Phase0: Remote Keyless Entry feature file with Gherkin scenario.

The UX requirements are derived in the Gherkin syntax, and a feature file is created, illustrated in Figure 4. The feature file reduces ambiguity in the requirements, by having a structured and standardized syntax. Next, a step definition file which has unit functions for each step in the feature file and a b-program is implemented.

Next, scenarios are executed over the Simulink model with behavioural composition executing and verifying the outputs. This enabled to identify any functional contradictions at early stage especially for critical safety requirements.

Overall, Phase 0, simulation over the model verified requirements before starting formalised feature models. Some of the test results are illustrated in Figure 5 confirm business-level scenarios passed and no tests are skipped, verifying the requirements syntax and logic before moving to Phase 1 technical design.

```

Sending RemoteKeylessEntry_Feature_Copy/Crash to model with value 1 and type <class 'int'>
BP events: ['CRASH_SIGNAL', 'UNLOCK_DOORS']
When the vehicle has "yes" # steps/lockingstep.parameterized.py:99 0.800s
Then all the doors should be UNLOCKED # steps/lockingstep.parameterized.py:161 0.271s
Given the vehicle crash status "no" # steps/lockingstep.parameterized.py:99 0.264s<class 'str'>
When the user has pressed the lock button # steps/lockingstep.parameterized.py:141 0.280s
Then all the doors should be LOCKED # steps/lockingstep.parameterized.py:161 0.262s
Scenario finished: After crash clears, manual lock works again with value All_Doors_Locked and type <class 'str'>

All tests done!!!
1 feature passed, 0 failed, 0 skipped
11 scenarios passed, 0 failed, 0 skipped
65 steps passed, 0 failed, 0 skipped
Took 0min 21.311s

```

Figure 5: Phase 0: Test results of automated simulation. Scenarios with behavioural program (BP events) being executed during unlocking event over the Simulink model in Figure 3.

4.2 Phase 1: Feature System Design

After Phase-0 requirements verification, Phase1 feature system design is started. Phase1 involved a formal SysML approach to describe system elements and functional decomposition. A feature system architecture is defined with a structural model and then behavioural modeling using state machines.

4.2.1 Structural Modeling

Using SysML tool structural model is created to define the structural architecture of the feature 'RKE'. An End-to-End (E2E) feature boundary diagram is created with interfaces, capabilities, and some of the key elements are as below –

- User Inputs (Key Fob button states, SPE/MPE configuration)
- Vehicle state inputs (Speed, Crash sensor, Battery-state-of-charge)
- Output blocks for each door latch
- Supporting service configuration

All signals are enumerated in a structured library to ensure standardised naming and semantic consistency across behavioural diagrams and tests.

4.2.2 Feature Decomposition

From use case modeling, the RKE feature is decomposed into two primary system functions: Door Unlocking, and Door Locking. Each function has its own dedicated E2E Gherkin feature files. These scenarios prepared the model for executable scenario simulation.

4.2.3 Behavioural Modeling – State Machine

Two primary state machine diagrams were created to capture door-locking and door-unlocking logic: Unlocking State Machine incorporated transitions for SPE and MPE behaviour. Locking state machine has composite state machine to minimise redundancy and streamline logic representation.

Both state machines mirrored the logic established in Phase 0 but provided a more formalised and traceable representation within the MBSE toolchain.

4.3 Phase1: Simulation

For this, MagicDraw2024x tool MBT plugin with Python is implemented, and Gherkin scenarios are executed over the state machines. The Behave runner is set up to communicate with the MagicDraw2024x API (Application Programming Interface) to trigger transitions in the State Machines.

4.3.1 Executing Scenarios

Both locking and unlocking scenarios are executed with consistency check over the state machines:

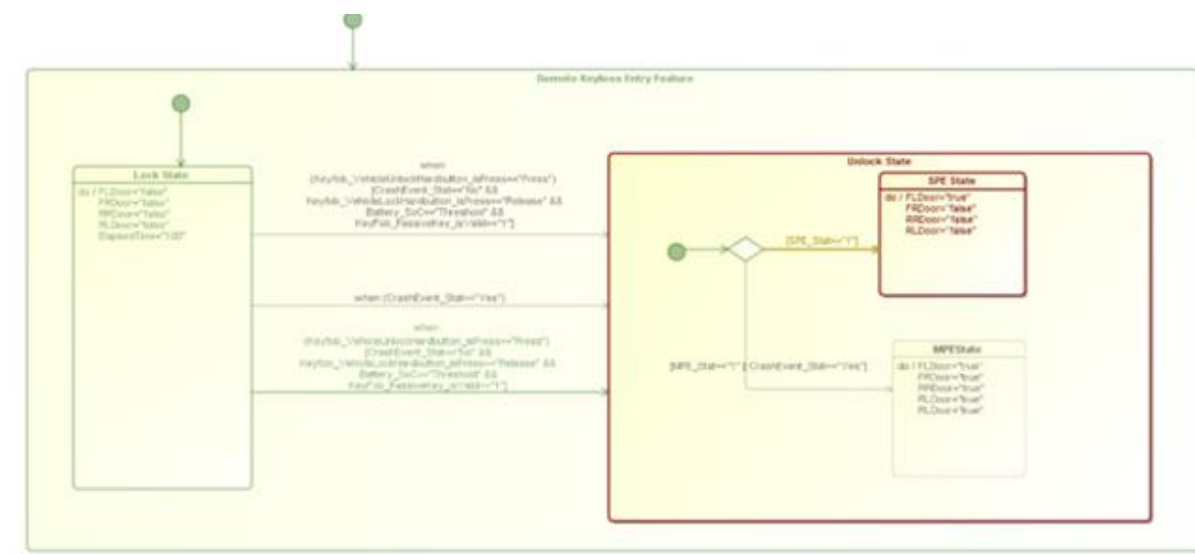


Figure 6: Phase 1: Remote Keyless Entry Feature Unlocking function scenarios automated execution over function state machine. In this, FL: Front Left, FR: Front Right, and RR: Rear Right.

- The locking scenarios validated state machine transitions involving driver lock requests, multipoint lock behaviour, auto locking above threshold speed, and crash override behaviour.
- The unlocking scenarios validated SPE/MPE configuration to verify unlock driver door when SPE selected and unlock all doors MPE is selected, and crash signal priority to unlock all doors. This is illustrated in Figure 6.
- Early Error Detection: Ambiguities in the “Auto-Lock” requirement were discussed and identified by stakeholders and fixed during the Simulink phase (Phase0) rather than at later system or vehicle testing.
- Machine Executable: Gherkin scenarios served as both documentation and automated test scripts, ensuring that the design meets the specification.
- Traceability: Tags in the BDD feature files established a robust link between different level of requirements and specific simulation results, and traceability standards.

4.3.2 Test Coverage Analysis

The test report illustrated in with simulation results. The test report highlighted that state machines responded correctly to scenario triggers, with all transitions firing as expected and no test steps were skipped.

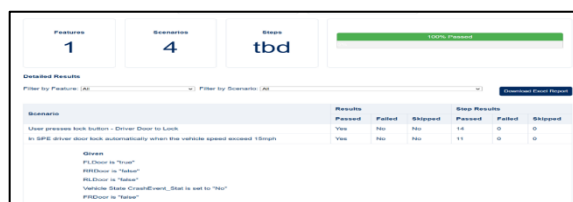


Figure 7: Phase1: MagicDraw2024x Test Report capturing test results and details.

4.4 Analysis

The test report analysis demonstrated that the integrated framework provides several advantages

5 CONCLUSIONS AND FUTURE WORK

The paper presented an integrated framework that effectively verifies the requirements during the early phases (Phase0 and Phase1). The results demonstrate significant advantages over feature system requirements written in standard non machine executable format.

Key benefits over standard MBSE methodologies include –

- Shift-Left verification: Enabling upfront verification of requirements, thereby identifying defects earlier in the V-cycle
- Early behavioural validation: The framework enables the specification and upfront verification of complex behaviours.

Additionally, it is possible for addition of new test cases without affecting the existing architectural model.

- Reduced ambiguity: Eliminating requirement errors and semantic gaps by implementing logic within machine executable specification.

The framework provides a structured architecture that manages the complexity of scenario-based verification from beginning. Moreover, it facilitates robust requirements verification upfront and supports a seamless transition to virtual/hardware system validation like HiL, MiL, and other system testing.

This case study presented Phase 0 and Phase1, demonstrating that subsequent phases can also be realised using the same integrated framework.

Future research work will investigate the Large Language Models (LLMs) to generate more Gherkin scenarios with edge cases, to assess the usefulness for SysML model improvements and for robust verification upfront.

ACKNOWLEDGEMENTS

There are no conflicts between the authors. Thank you to Ford Motor Company, and Warwick University, WMG, for resources, reviews and technical support.

REFERENCES

- Ali Rizvi, Hepp, D., Jana, P., Buchner, F., Gebler, F., & Feike, J. (2025). Winning the automotive software development race (Automotive & Assembly Practice) [White Paper]. McKinsey & Company. <https://www.mckinsey.com/~media/mckinsey/industries/automotive%20and%20assembly/our%20insights/winning%20the%20automotive%20software%20development%20race/winning-the-automotive-software-development-race.pdf?shouldIndex=false>
- Apostu, S., Burkacky, O., Deichmann, J., & Doll, G. (2019). Automotive software and electrical/electronic architecture: Implications for OEMs. McKinsey Center for Future Mobility. <https://www.mckinsey.com/~media/mckinsey/industries/automotive%20and%20assembly/our%20insights/automotive%20software%20and%20electrical%20electronic%20architecture%20implications%20for%20oems/automotive-software-and-electrical-electronic-architecture-implications-for-oems.pdf?shouldIndex=false>
- Automotive software and electronics 2030. (2030).
- Bajer, M., Szlagor, M., & Wrzesniak, M. (2015). Embedded software testing in research environment. A practical guide for non-experts. 2015 4th Mediterranean Conference on Embedded Computing (MECO), 100–105. <https://doi.org/10.1109/MECO.2015.7181877>
- Dr Aktas, I. (2023). Trends towards Software-defined Vehicles [White Paper]. ETAS. https://www.linkedin.com/posts/chris-schill-2973ab2_trends-towards-software-defined-vehicles-activity-7097196336923488256-wj71
- Gill, Dr. R. (2022). A Study of Framework of Behavioural Driven Development: Methodologies, Advantages, and Challenges. *International Journal on Future Revolution in Computer Science & Communication Engineering*, 8(2), 09–12. <https://doi.org/10.17762/ijfrcsce.v8i2.2068>
- Graessler, I., & Hentze, J. (2020). The new V-Model of VDI 2206 and its validation. *At - Automatisierungstechnik*, 68(5), 312–324. <https://doi.org/10.1515/auto-2020-0015>
- Harel, D., Marron, A., & Weiss, G. (2012a). Behavioral programming. *Communications of the ACM*, 55(7), 90–100. <https://doi.org/10.1145/2209249.2209270>
- Harel, D., Marron, A., & Weiss, G. (2012b). Behavioral programming. *Communications of the ACM*, 55(7), 90–100. <https://doi.org/10.1145/2209249.2209270>
- ISO 26262-1:2018. (2018, December). ISO. <https://www.iso.org/standard/68383.html>
- Júnior, F. S., Reis, P. A., Cavalcante, M. S., & De Oliveira, A. H. M. (2024). Systems Engineering Process Enhancement: Requirements Verification Methodology using Natural Language Processing (NLP) for Automotive Industry. 2023-36-0117. <https://doi.org/10.4271/2023-36-0117>
- Petrenko, A., Simao, A., & Maldonado, J. C. (2012). Model-based testing of software and systems: Recent advances and challenges. *International Journal on Software Tools for Technology Transfer*, 14(4), 383–386. <https://doi.org/10.1007/s10009-012-0240-3>
- Reference | Cucumber. (2025, January 26). <https://cucumber.io/docs/gherkin/reference>
- Staron, M. (2019). Requirements Engineering for Automotive Embedded Systems. In Y. Dajsuren & M. van den Brand (Eds), *Automotive Systems and Software Engineering: State of the Art and Future Trends* (pp. 11–28). Springer International Publishing. https://doi.org/10.1007/978-3-030-12157-0_2
- Utting, M., & Legeard, B. (2007). *Practical Model-Based Testing. A Tools Approach*. Elsevier. <https://0-www-sciencedirect-com.pugwash.lib.warwick.ac.uk/book/9780123725011/practical-model-based-testing>
- Wiecher, C., Japs, S., Kaiser, L., Greenyer, J., Dumitrescu, R., & Wolff, C. (2020). Scenarios in the loop: Integrated requirements analysis and automotive system validation. *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*, 1–10. <https://doi.org/10.1145/3417990.3421264>
- Zameni, T., Van Den Bos, P., Tretmans, J., Foederer, J., & Rensink, A. (2023). From BDD Scenarios to Test Case Generation. 2023 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), 36–44. <https://doi.org/10.1109/ICSTW58534.2023.00019>
- Zander, J., Schieferdecker, I., & Mosterman, P. J. (Eds). (2017). *Model-Based Testing for Embedded Systems*. CRC Press. <https://doi.org/10.1201/b11321>